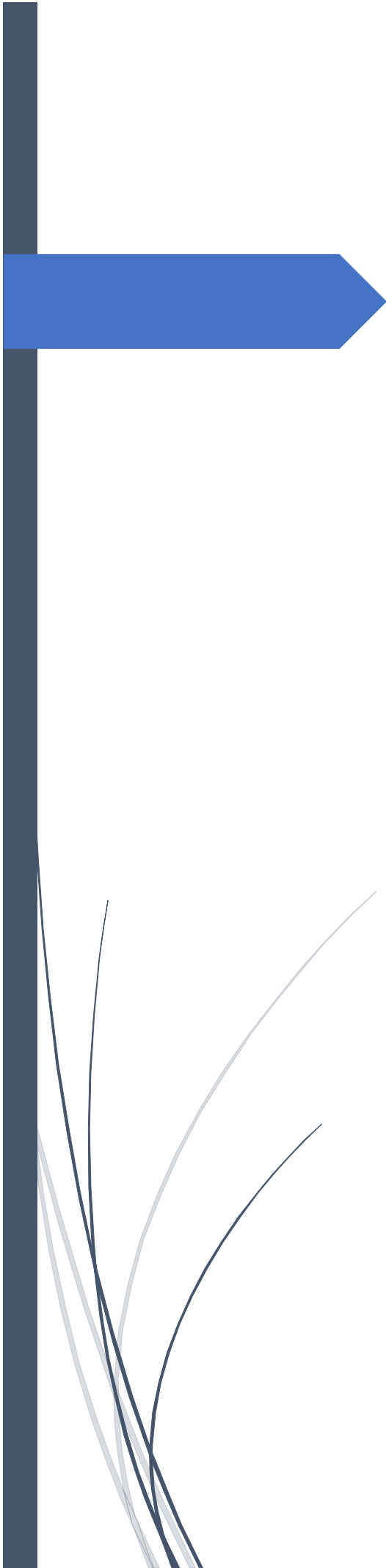




ISPITNA PITANJA

Objektno-orijentisano programiranje

Jefimija Stamenović

Sadržaj

I.	Motivacija	2
II.	Osnovni koncepti objektnog programiranja i njihova implementacija u Javi.....	3
III.	Nasleđivanje (uopšte o pojmovima vezanim za nasleđivanje).....	6
IV.	GUI.....	13
V.	Niti.....	17
VI.	OOP paradigma i UML.....	19

I. Motivacija

1. Objasniti pojam i dati primer – ADT – Abstract Data Type

ADT (Abstract Data Type) tj. apstraktni tip podatka je tip podatka čiju implementaciju ne znamo, ali znamo kako se ponašaju operacije nad vrednostima tog tipa i u skladu sa tim pišemo program.

Za pojam ADT je neraskidivo vezan i **pojam sakrivanja informacije (information hiding)**. Ovaj pojam znači da nam se daje ime tipa i paket procedura. Implementacija tipa se sakrije tako da je mi ne vidimo.

Primer: Želimo da radimo sa tipom podatka *matrica*. Imamo naziv tipa *matrix* i operacije za taj tip: *množenje, deljenje, transponovanje, sabiranje, oduzimanje...* Međutim, mi ne znamo da li je tip matrice implementiran kao niz ili preko pokazivača.

2. Objektno spram strukturnog modelovanja

Strukturno modelovanje se zasniva na analizi problema i razbijanja problema na potprobleme. Svaki potproblem se može implementirati u okviru jedne funkcije. Funkcija je osnovni gradivni oblik programa pa program predstavlja niz funkcijskih poziva. Podaci se smeštaju u strukture koje često ne predstavljaju realnu sliku sveta. Program počinje izvršavanjem funkcije `main()` koja poziva druge funkcije, a završava se kada se izvrše sve funkcije napisane u okviru `main-a`.

Objektno modelovanje se zasniva na razmatranju kompletnog sistema u kome treba da se obavi posao. Uočavaju se učesnici i njihove aktivnosti koje obavljaju sa određenim podacima. Osnovnu ulogu imaju objekti koji sadrže podatke i funkcije (metode). Podaci predstavljaju stanje objekta, a pomoću metoda, to stanje može da se promeni i objekat može da komunicira sa drugim objektima. Program se konstruiše kao skup objekata koji međusobno komuniciraju.

3. Karakteristike Java jezika

- 1) **Objektna orijentacija** – podržava sve koncepte objektno-orijentisanog programiranja, a sintaksa je slična C++, s tim što je objektno-orijentisani model jednostavniji
- 2) **Prenosivost** – Java programi se prevode u byte kod koji nije mašinski jezik nijednog konkretnog računara, već se izvršava na JVM (Java Virtuelna Mašina).
- 3) **Sigurnost** – JVM pruža zaštitu od virusa koji bi se prenosili kroz izvršni kod
- 4) **Robusnost** – Stroga provera tipova, proveravani izuzeci, sakupljanje đubreta
- 5) **Efikasnost** – JIT (Just In Time) prevodioci

4. Izvršavanje Java programa

1) Bajtkod

Java bytecode je mašinski kod za JVM.

2) JVM (Java Virtual Machine)

JVM je interpreter koji izvršava Java bytecode

3) Komande

`javac naziv-klase.java` – prevodilac kreira datoteku `naziv-klase.class` koja sadrži bytecode instrukcije za JVM

`java naziv-klase` – pokreće se program pozivanjem JVM uz prosleđivanje bajtkoda

4) JRE (Java Run-time Environment)

JRE je okruženje koje je neophodno za izvršavanje java programa. Sadrži - JavaVM i neophodne biblioteke.

II. Osnovni koncepti objektnog programiranja i njihova implementacija u Javi

5. Objasniti pojmove objekta i klase i dati primer (nezavisno od Jave)

Klasa predstavlja implementaciju tipa, a objekat je jedan primerak tog tipa.

Primer: Klasa – STUDENT, Objekat – Petar Petrović

6. Definicija klase u Javi

```
<modifikator> class <className> {  
  
    <modifikator> <tip> <imepromenljive1>;  
    <modifikator> <tip> <imepromenljive2>;  
    ...  
  
    <modifikator> <povratni tip> <imemetoda1> (<tip> <arg1>,...) {  
        ... //implementacija metoda 1  
    }  
    <modifikator> <povratni tip> <imemetoda2> (<tip> <arg1>,...) {  
        ... //implementacija metoda 2  
    }  
    ...  
}  
// kraj definicije klase
```

Diagram illustrating the structure of a Java class definition:

- podaci članovi** (class members/variables):
 - <modifikator> <tip> <imepromenljive1>;
 - <modifikator> <tip> <imepromenljive2>;
- funkcije članice - metodi** (class methods):
 - <modifikator> <povratni tip> <imemetoda1> (<tip> <arg1>,...) {
 ... //implementacija metoda 1
}
 - <modifikator> <povratni tip> <imemetoda2> (<tip> <arg1>,...) {
 ... //implementacija metoda 2
}

7. Opisivanje stanja i ponašanja objekata. Dati primer (nezavisno od Jave)

8. Kreiranje i upotreba objekata u Javi. Objasniti i dati primer.

```
Light lt = new Light();
```

```
lt.on();
```

lt – predstavlja referencu na objekat tipa Light

new Light() – kreira se objekat tipa Light u memoriji

lt.on() – slanje poruke objektu lt

9. Komunikacija sa objektima. Pristup podacima i slanje poruka (nezavisno od Jave)

10. Metodi. Definisanje metoda u Javi. Opisati i dati primer. Deklaracija i definicija (implementacija) metoda.

Kod definisanja metoda u Javi, obavezni su : povratni tip, ime metode i ()

```
<modifikator> <povratni tip> <ime_metode>(<argumenti>){
```

```
// implementacija metode
```

```
....
```

```
}
```

```
String s = a.toString ();
```

```
float naturalLongBase { return 2.718f; }
```

```
void nothing(){return; }
```

11. Pojam konstruktora. Konstruktori u Javi.

1) Navesti karakteristike konstruktora u Javi. Dati primere.

2) Pitanje postojanja i izvršavanja konstruktora

Konstruktor je specijalna vrsta metoda koji se koristi isključivo pri konstruisanju objekata.

Karakteristike:

1) Ima isto ime kao i klasa

2) Poziva se isključivo pri instanciranju objekata (operatorom new)

3) Nema povratne vrednosti

Klasa može imati više konstruktora. Konstruktor bez parametara je **default konstruktor** i on postoji i kada klasa nema posebno naveden konstruktor, a u suprotnom neće postojati.

```
class Tacka{
    private double x;
    private double y;
    public Tacka() { x=0.0; y=0.0; }
    public Tacka(double a, double b) { x=a; y=b; }
    public double getX() { return x; }
    public double getY() { return y; }
}

public class Test {
    public static void main(String[] args){
        Tacka a = new Tacka();
        Tacka b = new Tacka(1,1);
        System.out.println("A(" + a.getX() + "," + a.getY() + ")\n");
        System.out.println("B(" + b.getX() + "," + b.getY() + ")\n");
    }
}
```

konstruktori

12. Overloading. Objasniti i dati primere.

Overloading je mogućnost definisanja više metoda sa istim imenom, ali različitim potpisom u jednoj klasi ili prilikom nasleđivanja. Istoimeni metodi se moraju razlikovati po broju ili bar po tipu argumenata. Dozvoljeno je da vraćaju različite tipove, ali pod uslovom da se razlikuju po argumentima.

Primer:

```
public class test {
    public static void main(String[] args) {
        Broj b = new Broj();
        b.uvecaj(1); // 1
        b.uvecaj(1.0); // 2
        b.uvecaj(1,1); // 3
        b.uvecaj(1.0, 2.0); // ?
    }
}

class Broj
{
    int vrednost = 10;
    int uvecaj(int i) { return vrednost + i; } // 1
    double uvecaj(double i) { return vrednost + i; } // 2
    double uvecaj(float i, float j) { return vrednost + i + j; } // 3
}
```

13. Oblast važenja (scope) imena u Javi. Stack, heap, Garbage Collector

- Inicijalizacija automatskih promenljivih je obavezna, a oblast važenja je slična kao i u C-u s tim što ne mogu da postoje dve promenljive sa istim imenom bez obzira što su u različitim blokovima.

```
{ int x = 12;
    /* samo je x dostupno */
    { int q = 96;
        /* x i q su dostupni */
    }
    /* samo x je dostupno a q 'ne postoji' */
}

{ int x = 12;
    {
        int x = 96; /* nepravilno */
    }
}
```

Životni vek objekata nije isti kao životni vek primitivnih tipova. Nakon kreiranja, objekat postoji i posle } iako je njegova referenca izašla iz oblasti važenja.

- Stack (stek)** je deo memorije koji je rezervisan za čuvanje lokalnih promenljivih, parametara procedura, povratnih adresa itd. Na steku se pakuju samo reference objekata, ali ne i objekti.

- **Heap (hrpa)** je deo memorije koji je rezervisan za dinamičke promenljive tj, za promenljive koje se stvaraju u toku izvršavanja programa.
- **Garbage Collector** je posebna nit koja vodi računa i uklanja objekte kada više ne postoji nijedna referenca na njih. Ona se izvršava kada to dozvoli procesorsko vreme ili kada je neophodno osloboditi memoriju.

14. this i this()

This je implicitni parametar koji sadrži adresu objekta čiji se metod poziva

this() je poziv default konstruktora vlastite klase, a **this(<arguments>)** je poziv odgovarajućeg vlastitog konstruktora. Ako ovaj poziv postoji, on mora biti prva naredba u konstruktoru.

15. Članovi objekata i članovi klase

1) Podaci

2) Metodi

Objasniti i dati primere definisanja i upotrebe

- **Članovi klase**

To su statički podaci i statičke funkcije. Ispred deklaracije ovih promenljivih i metoda se mora navesti modifikator **static**. Static modifikator obeležava zajedničke članove svim instancama date klase.

```
class Test {
    int x;           // nestatička javna varijabla
    static int y;

    public void print() { // nestatička metoda
        System.out.println("Hello, World");
    }
    public static void print_static() { // statička metoda
        System.out.println("Hello, World");
    }
}
```

Za korišćenje statičkih metoda nije potrebna instanca te klase. Može im se pristupiti sa:

(ime_klase).(ime_statičke_metode)

Statičke metode ne mogu da koriste nestatičke metode iste klase direktno, niti mogu da pristupe nestatičnim promenljivama. Razlog za ovo je to što nestatičke promenljive sadrže parametar *this*, a statičke ne.

- **Članovi objekata**

To su nestatički podaci i nestatički metodi koje poseduju svi objekti date klase i predstavljaju pojedinačno stanje i aktivnosti objekta. Svaki objekat implicitno ubacuje parametar *this* kojim prosledjuje svoju adresu.

III. Nasleđivanje (uopšte o pojmovima vezanim za nasleđivanje)

16. Klasifikacija, generalizacija i specijalizacija

Klasifikacija je proces određivanja i svrstavanja objekata sa istim karakteristikama u klase.

Generalizacija je postupak pronalaženja sličnosti između klasa/apstrakcija i definisanja sličnosti u novoj generalizovanoj klasi

Specijalizacija je proces određivanja razlika među objektima jedne klase i definisanja novih potklasa koje sadrže razlike

17. Objasniti pojam nasleđivanja. nadklasa, podklasa

Nasleđivanje je mogućnost uvođenja novih tipova/klasa proširivanjem osobina i ponašanja postojećih tipova/klasa.

Klasa koja nasleđuje tj. proširuje se naziva IZVEDENA, POTKLASA ili **DETE-KLASA** i predstavlja užu tip podatka u odnosu na klasu iz koje je izvedena.

Klasa koja biva proširena tj. čije osobine i ponašanja nasleđuje dete-klasa se naziva **SUPERKLASOM**, **NATKLASOM** ili **RODITELJSKOM KLASOM** i predstavlja širi tip podatka u odnosu na klasu iz koje je izvedena. Svaka klasa može da ima neograničen broj potklasa. Potklase nisu ograničene na promenljive, konstruktore i metode klase iz koje su izvedene i mogu da dodaju i neke druge promenljive i metode ili predefinisati stare metode.

```
class B extends A {  
    //telo klase B  
}
```

18. Jednostruko i višestruko nasleđivanje

U Javi, klasa može da ima samo jednu natkласu. Sve klase u Javi su direktno ili indirektno izvedene iz klase **Object**. Ako se eksplicitno ne navede natklasa neke klase, onda je ona implicitno izvedena iz **java.lang.Object**

19. Enkapsulacija. Vidljivost podataka i metoda

Enkapsulacija (učaurivanje) predstavlja mehanizam sakrivanja informacija i izvodi se upotrebom **modifikatora vidljivosti/pristupa**

Modifikatorima vidljivosti se određuje opseg dostupnosti elemenata koda (polja, metoda, tipova). Svaki element ima definisanu vidljivost (implicitno ili eksplicitno). Objekat potklase uvek sadrži kompletan objekat superklase, ali to ne znači da su svi članovi superklase dostupni metodama koje su specifične samo za klasu.

20. Polimorfizam

Polimorfizam je mogućnost da varijablom određenog tipa referenciramo objekte različitih tipova i da automatski pozivamo metode koje su specifične za tip objekta koji varijabla referencira.

Uslovi:

- 1) Poziv metoda potklase kroz varijablu bazne klase
- 2) Pozvana metoda mora biti član bazne klase
- 3) Signatura metode i povratni tip moraju biti isti u baznoj i u izvedenoj klasi
- 4) Atribut pristupa ne sme biti restriktivniji u izvedenoj klasi nego što je u baznoj klasi
- 5) Polimorfizam se odnosi samo na metode – reference baznog tipa mogu se koristiti samo za pristup podacima članovima baznog tipa

21. Nasleđivanje u Javi. Objasniti i dati primere

1) Konstrukcija objekata u Javi. Super()

Kada se kreira objekat, njegovi atributi se postavljaju na podrazumevanu vrednost: Nula za numeričke, false za boolean i null za reference. Nakon inicijalizacije, prvo se poziva konstruktor superklase, pa zatim telo konstruktora klase kojoj objekat koji se kreira pripada. Izvedena klasa **mora da pozove jedan od konstruktora super klase**.

Eksplícitan poziv konstruktora super klase se poziva sa super (argumenti). Ako se konstruktor super klase eksplicitno poziva, onda taj poziv mora biti prva komanda u konstrukturu izvedene klase. Ako se ne poziva eksplicitno, implicitno će biti pozvan default konstruktor natklase.

```
class A {
    int aData;
    A() { // super();
        System.out.println("Pozvao A()");
    }
}
class B extends A {
    B() { // super();
        System.out.println("Pozvao B()");
    }
}
```

```
class C extends B {
    int cData;
    // C() { super(); }
    C() { // super();
        System.out.println("Pozvao B()");
    }
}
class Test{
    public static void main(String[]
args){
        C c = new C();
        System.out.println(c);
    }
}
```

```
class A {
    int aData;
    A(int a) { aData = a;}
}
class B extends A {
    B() {
        super(10);
        System.out.println("Pozvao B()");
    }
}
```

```
class C extends B {
    int cData;
    public String toString() { return ""+aData; }
}
class Test{
    public static void main(String[] args){
        C c = new C();
        System.out.println(c);
    }
}
```

2) Prepisivanje metoda (overriding)

Overriding je termin koji se koristi kada se u potklasi definiše metoda sa istim imenom, povratnom vrednošću i argumentima kao i metoda superklase čime se prepisuje metod iz superklase.

22. Kompatibilnost tipova i konverzija uopšte u Javi. Objasniti i dati primere

Java je strogo tipiziran jezik. Kompatibilnost tipova se proverava tokom prevođenja. Pri operaciji dodele, inicijalizacije, prenosu parametara i vraćanju rezultata metoda, tip izraza mora biti kompatibilan tipu promenljive kojoj se izraz dodeljuje. Tip reference koja je rezultat izraza mora biti: tip promenljive kojoj se dodeljuje, njegov podtip ili null.

Konverzija

Nadtipovi su manje posebni, smatraju se širim i nalaze se u više hijerarhija tipova. Imamo dve vrste konverzije:

- 1) Konverzija **proširivanja (naviše, nagore)** – podtip se konveruje u nadtip, vrši se automatski
- 2) Konverzija **sužavanja (naniže, nadole)** – nadtip se konvertuje u podtip, vrši se eksplicitno (kastovanje)


```

class Student extends Gradjanin { ... }
...
Student s = new Student();
Gradjanin g = new Gradjanin();
Gradjanin g2 = s;      // naviše
Student s1 = (Student) g; // naniže, obavezan cast

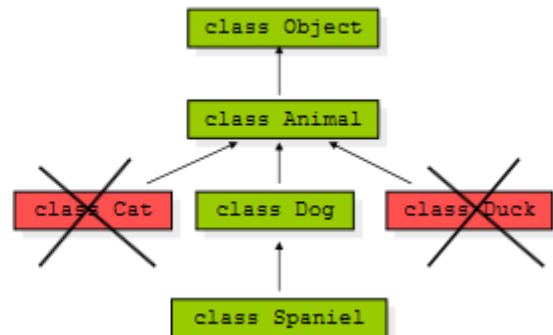
```

Cast je moguć u slučaju da radimo cast između klasa koje pripadaju istom lancu nasleđivanja – jedan tip mora da bude podtip drugog.

```

Spaniel aPet = new Spaniel("Fang"); // Kreiraj objekat tipa Spaniel
Animal theAnimal = (Animal)aPet;
Animal theAnimal = aPet;
    // cast nagore - upward cast
Dog aDog = (Dog)theAnimal;
    // cast nadole
    // downward cast

```



aDog može da koristi samo overridden metode iz klase *Spaniel*. Dodatne metode iz klase *Spaniel* neće biti dostupne sa aDog referencom osim ako se ne kastuje iz klase *Dog* u klasu *Spaniel*.

23. Statičko i dinamičko vezivanje (binding) poziva sa kodom koji treba da bude izvršen.

24. Paketi. Čemu služe i kako se definišu i koriste (u kodu i pri kompajliranju)

Paket je kolekcija srodnih tipova (klasa i interfejsa). Razlozi za korišćenje paketa su:

- 1) Naznačavanje srodnosti određenih tipova
- 2) Olakšavanje pronalaženja željenog tipa (fokusiranjem na samo jedan paket)
- 3) Otklanjanje potencijalnih duplikata u nazivima (jedinствeni prostor imena)
- 4) Kontrolisanje pristupa (klase u okviru istog paketa mogu da imaju neograničen pristup jedna drugoj, a spoljne ne)

```

package geometry;
public class Sphere {
    // Details of the class definition
}

```

package mora biti prva naredba u fajlu i u jednom fajlu i dozvoljena je deklaracija samo jednog paketa. Jedan tip može pripadati samo jednom paketu i ime tipa u okviru paketa je jedinstven.

Upotreba:

- 1) Navođenjem punog imena tipa : <ime_paketa>.<ime_tipa>

```

public class Ball {
    geometry.Sphere b = new geometry.Sphere();
    ...
}

```

- 2) Izvršiti uvoz tipa ili svih tipova paketa

```

import geometry.Sphere; // ili import geometry.*;
public class Ball {
    Sphere b = new Sphere();
    ...
}

```

Kompajler traži izvorni ili bajt kod u kome je definisan tip i to prvo traži u:

- 1) Tekućem folderu, pa u
- 2) Lib direktorijumu JRE-a, a zatim u
- 3) tzv. user class path-u tj. korisnički definisanim putanjama do korisničkih klasa.

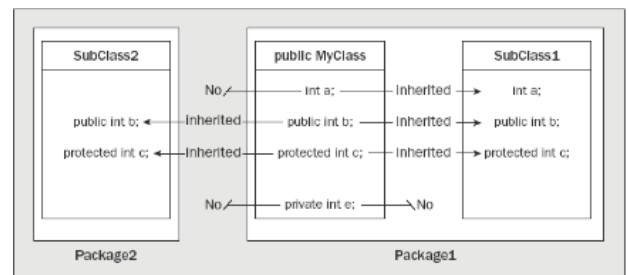
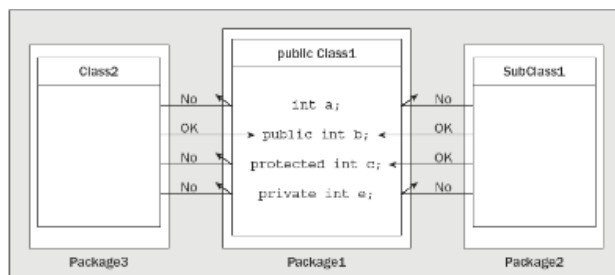
JAR Fajlovi

.jar – java archive

Jar archive sadrže kompresovane class fajlove sa kompletno sačuvanom direktorijumskom strukturom i obezbeđuju jednostavnost u prenosu i upotrebi većeg broja korisnički definisanih paketa

25. Modifikatori pristupa u Javi

Tip \ vidljivost	Dostupno (vidljivo)			
	klasi	podklasi	paketu	bilo kome
Private	X			
protected	X	X*	X	
public	X	X	X	X
Nothing(default) Package Friendly	X		X	



26. Final modifikator. Objasniti uticaj na podatke i metode. Pokazati opisano na primerima

Final modifikator je modifikator koji može biti primenjen na klase, promenljive i metode. Upotrebom ovog modifikatora se definicija elementa proglašava konačnom.

- 1) **final promenljiva** – varijabla predstavlja konstantu i ne menja vrednost. Inicijalizacija ne mora da bude obavljena na mestu gde je konstanta deklarirana, ali je vrednost moguće samo jednom postaviti.
- 2) **final klasa** – definicija tipa je konačna pa klasa ne može biti proširena tj. ne može biti nasleđena
- 3) **final metod** – ne može biti prepisan

Final može da bude i argument metoda i tada se ta varijabla ne može menjati unutar metoda.

```
class Calculator {
    final int dime = 10;
    int count = 0;
    Calculator (int i) { count = i; }
    void Inc(final int i) {
        i++; // illegal
    }
}

class RunCalculator {
    public static void main(String[] args) {
        final Calculator calc = new Calculator(1);
        calc = new Calculator(2); // illegal
        calc.count = 2;
        calc.dime = 11; // illegal
        System.out.println("dime: " + calc.dime);
    }
}
```

27. Super

Super je eksplicitni parametar pomoću kog možemo da pristupimo metodama i promenljivama superklase.

28. Object klasa

Svaka klasa definisana u Javi, direktno ili indirektno nasleđuje klasu Object. Klase koje ne koriste klauzulu **extends** implicitno proširuju klasu Object. Metodi klase Object su:

- 1) protected Object clone ()
- 2) boolean equals (Object obj)
- 3) protected void finalize()
- 4) Class <?> getClass ()
- 5) int hashCode()
- 6) void notify()
- 7) void notifyAll()
- 8) String toString()
- 9) void wait() sa još dve overloaded varijante

29. Apstraktni metodi. Apstraktne klase. Objasniti i dati opis načina definisanja kao i primer definicije jedne apstraktne klase i njene upotrebe

U apstraktni tipove spadaju **apstraktne klase** i **interfejsi**.

Apstraktni metodi su metodi koji nemaju implementaciju. Definišu se ključnom rečju **abstract**.

Apstraktne klase su klase koje ne mogu da se instanciraju. Klase su apstraktne ukoliko sadrže apstraktnu metodu ili ako je apstraktnost naglašena.

Karakteristike apstraktne klase:

- 1) Objedinjuje zajedničke karakteristike većeg broja klasa
- 2) Deklariše se upotrebom ključne reči **abstract**
- 3) Ne može biti instancirana, ali se može proširiti
- 4) Ako u nekoj klasi postoji apstraktna metoda, tada se i sama klasa mora definisati kao apstraktna. Klasa se može definisati kao apstraktna, a da pri tome nema nijedan apstraktan metod
- 5) Ukoliko potklasa neke apstraktne klase nema implementirane sve nasleđene apstraktne metode, onda se i potklasa mora proglasiti apstraktnom
- 6) Sama apstraktna klasa se ne može instancirati, ali se mogu instancirati konkretna deca-klase i mogu postojati reference tipa apstraktne klase koje ukazuju na objekte konkretnih klasa koje su izvedene iz apstraktne

Primer apstraktne klase:

```
abstract class GraphicObject {  
    int x, y;  
    ...  
    void moveTo(int newX, int newY) {  
        ...  
    }  
    abstract void draw();  
    abstract void resize();  
}
```

30. Interfejs. Objasniti pojam, način upotrebe. Dati primer

Interfejs je reference tip koji sadrži:

- 1) **Metode** koje su podrazumevano **public** i **abstract**
- 2) **Podatke** koje su podrazumevano **public**, **static** i **final**

Interfejs se ne može instancirati

Može biti proširen tj. može biti izveden jedan interfejs iz drugog

```
public interface MerljivoExt extends Merljivo{
    double obim();
}
```

Upotreba:

Klase implementiraju jedan ili više interfejsa čime je delimično omogućen mehanizam višestrukog nasleđivanja.

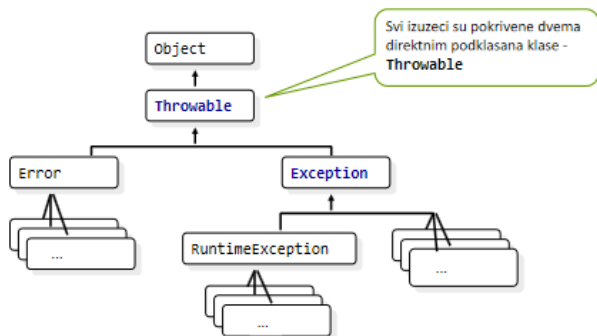
```
class A extends N implements interfejs1, interfejs2 {
    . . .
}
```

31. Izuzeci. Pojam. Definisanje u Javi. Proveravani i neproveravani izuzeci. Dati primere

Izuzeci su neregularne situacije koje utiču na rad programa (deljenje nulom, greške na resursima). Da bi se odvojio kod koji se izvršava u regularnoj situaciji od obrade neregularne situacije, u Javi se uvode:

- 1) Posebna vrsta upravljačkog bloka – try/catch/finally
- 2) Poseban mehanizam obaveštenja – instanciranje objekta specijalnog tipa i njegovo prosleđivanje onom delu koda koji je u stanju da ga obradi

Metod tokom čijeg se izvršenja desila neregularna situacija, zahteva od pozivaoca drugačije ponašanje, a obaveštava ga tako što generiše objekat specijalnog tipa (tipa **IZUZETKA**) i **baca ga** tj. prosleđuje ga komandi **throw**. JVM obrađuje throw tako što preusmerava izvršenje programa na deo koji je naveden u bloku za obradu odgovarajućeg tipa izuzetka tj. za **prihvatanje objekata** odgovarajućeg tipa (**catch blok**). Specijalan tip objekta je objekat kojim se opisuje greška i mora da pripada klasi koja se nalazi u lancu nasleđivanja klase **Throwable**.



Izuzeci tipa **Error** su izuzeci za koje se ne očekuje da ih hvata korisnik. Njene direktne potklase su:

ThreadDeath – baca se kada se namerno zaustavi nit u izvršavanju

LinkageError – ozbiljne greške unutar klasa u programu

VirtuaakMachineError – JVM greška

Za skoro sve izuzetke koji su obuhvaćeni potklasama klase **Exception** mora da se uključi kod za njihovu

obradu ili program neće proći kompajliranje tj. prevodilac ne dozvoljava njihovo ignorisanje.

RunTimeException izuzeci se tretiraju drugačije jer su posledica ozbiljnih grešaka u kodu čija obrada ne bi mogla ništa značajno da promeni. Prevodilac dozvoljava njihovo ignorisanje. Neke njene potklase su:

ArithmeticException, IndexOutOfBoundsException, NullPointerException

Na osnovu toga da li prevodilac insistira na njihovom proveravanju ili ne, izuzeci se dele u dve grupe:

- 1) **Proveravani izuzeci** – izvedeni su iz klase Exception i ne nalaze se u lancu nasleđivanja klase RuntimeException klase. Ako metod baca neki proveravani izuzetak, poziv te metode mora da bude uokviren try-catch blokom koji hvata taj izuzetak, a metoda mora da bude označena ključnom reči **throws** i nazivom klase izuzetka koji baca
- 2) **Neproveravani izuzeci** – izvedeni su iz klase RuntimeException. Ako metoda baca neki neproveravani izuzetak, poziv te metode može, a i ne mora da bude uokviren **try/catch** blokom koji hvata taj izuzetak.

```

public class Izuz extends RuntimeException{
    ...
}
public class KK {
    public void gr(int k) {
        if (k==0) throw new Izuz();
        // regularno ponasanje metoda
    }
}
public class TestIzuz {
    public static void main(String args[]) {
        KK k=new KK();
        k.gr(0);
        System.out.println("Regularan kraj programa!");
    }
}

```

U ovom primeru bačeni izuzetak nije obrađen, pa će program završiti sa radom pre ispisa Regularan kraj programa!

```

public class Izuz extends Exception{
    ...
}
public class KK {
    public void gr(int k) {
        if (k==0) throw new Izuz();
        // regularno ponasanje metoda
    }
}
public class TestIzuz {
    public static void main(String args[]) {
        KK k=new KK();
        k.gr(0);
        System.out.println("Regularan kraj programa!");
    }
}

```

Dve greške koje registruje kompajler

Finally blok

Koristi se za pospremanje

Asociran je sa određenim try i catch blokom

Može se koristiti i sa try blokom koji sadrži kod koji ne baca nikakav izuzetak

- 1) Za kod sa višestrukim break ili return elementima
- 2) Vrednosti koje vratimo sa return u finally bloku će pregaziti bilo koji return izvršen u try bloku.

```

int metod(){
    try {
        //...
        return 1;
    }
    finally {
        return 2;
    }
    // nedohvatljiv deo koda, kompajler ne bi dozvolio
}

```

Metod iz primera vraća 2.

IV. GUI

32. Grafičke komponente awt i swing paketa

1) Kontejneri

Kontejner je izvedena iz klase **Component** i objekat klase Container može da sadrži druge komponente. On je osnovno sredstvo za organizovanje komponenti. Svaki kontejner sadrži: **komponente** i definisanog **menadžera postavke**

2) Menadžeri postavki

Menadžer postavke je objekat neke od klasa tipa **Layout Manager**. On sadrži opis načina raspoređivanja komponenti u kontejneru. Klase menadžera postavke u awt paketu su: FlowLayout, BorderLayout, CardLayout, GridLayout, GridBagLayout, a u paketu swing: BoxLayout i SpringLayout

3) Komponente. Upotreba i primeri.

Komponente su grafički entiteti odgovarajućeg tipa koje mogu biti prikazane na ekranu i u Javi predstavljaju objekat bilo koje klase koja je potklasa klase **Component**. Klasa **Component** je zajednička osnovna klasa za sve GUI kontrole i reprezentuje nešto što: ima poziciju, veličinu, može da se iscrta na ekranu i prihvata ulazne događaje.

```
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JButton;
import javax.swing.JLabel;

import java.awt.Component;
public class TryWindow {

    public static void main(String[] args) {
        JFrame aWindow = new JFrame("This is the window Title");
        int windowWidth = 400; // window width in pixels
        int windowHeight = 150; // window height in pixels
        aWindow.setBounds(50, 100, windowWidth, windowHeight);
        aWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        aWindow.setVisible(true); // Display the window
        aWindow.getContentPane().setLayout(new java.awt.FlowLayout());

        JButton b = new JButton("Gumb 1");
        JButton b2 = new JButton("Gumb 2");
        JLabel label = new JLabel("Hello World, again!");

        String[] petStrings = { "Bird", "Cat", "Dog", "Rabbit", "Pig" };
        JComboBox petList = new JComboBox(petStrings);
        petList.setSelectedIndex(4);

        aWindow.getContentPane().add(label);
        aWindow.add(b);
        aWindow.add(b2);
        aWindow.add(petList);
        aWindow.setVisible(true);

        Component[] theComponents = aWindow.getContentPane().getComponents();
        for(Component component : theComponents) {
            System.out.println(component.getClass().getName()+"\n");
        }
    }
}
```

33. Ugnježdene klase. Anonimne klase. Objasniti i dati primer

Ugnježdeni tipovi su tipovi čija je definicija ugnježdena unutar definicije drugog tipa.

```
class Spoljasnja_klasa {
    // variables and methods for the outer class
    ...
    class Ugnjezdena_klasa {
        // variables and methods for the nested class
        ...
    }
}
```

Definicija tipa (klase ili interfejsa) se može ugnjezditi u:

- 1) telo metoda ili bilo kog bloka u kodu {...}
- 2) Definiciju klase ili interfejsa

Ugneždavanje u metod ili blok

```
class MyTopLevel{
    private String top = "From Top level class";
    public void createNestedInMethod() {
        class Prava {
            int i=1;
            public void getPrava(){System.out.println(i);}
        }
        Prava p=new Prava();
        { class NePrava{
            int j=1;
        }
        NePrava np=new NePrava();
        NePrava np=new NePrava();
    }
}
```

ugneždjeno u metod

ugneždjeno u blok

Lokalne klase

Lokalna klasa je klasa definisana unutar proizvoljnog programskog bloka (metode, konstruktora, inicijalizacionog bloka). Ona nije članica okružujuće klase i nepristupačne su van bloka u kojem su definisane. Instance lokalne klase su normalni objekti koji mogu da se prenose kao argumenti ili da se vraćaju kao rezultat metoda. Iz okružujućeg bloka, lokalna klasa ima pristup samo *final lokalnim varijablama* i *final argumentima metoda*

```
public interface Iterator{
    boolean hasNext();
    Object next() throws
        NoSuchElementException;...
}
```

Klasa Iter je lokalna klasa, klijenti metoda obilazak() nisu svesni tipa Iter.

```
public static Iterator obilazak(final Object[] objekti){
    class Iter implements Iterator{
        private int p = 0; //pozicija u nizu objekti
        public boolean hasNext() {return (p<objekti.length);}
        public Object next() throws NoSuchElementException {
            if (p>=objekti.length) throw new
                NoSuchElementException();
            return objekti[p++];
        }
    }
    return new Iter();
}
```

Anonimna klasa

Anonimna klasa je posebna vrsta lokalne klase koja se instancira na mestu gde se daje njihova definicija, pa im se ne navodi ime. Anonimna klasa proširuje drugu klasu ili implementira neki interfejs. Definiše se u izrazu kao deo naredbe za instanciranje nadklase/interfejsa koji anonimna klasa proširuje/implementira. Iako proširuje/ implementira, ne koristi eksplicitno klauzule *extends* i *implements*. Ne može da ima konstruktor. Ne mogu da pristupaju lokalnim varijablama i argumentima ako nisu final, ali **mogu da pristupe podacima okružujuće klase**.

```
public static Iterator obilazak(final Object[] objekti){
    return new Iterator(){
        private int p = 0; //pozicija u nizu objekti
        public boolean hasNext(){
            return p<objekti.length;
        }
        public Object next() {
            return objekti[p++];
        }
        public void remove() { // TODO }
    };
}
```

Ugneždavanje u definiciju tipa

Top-level tip je tip koji sadrži u sebi ugneždeni tip, a sam po sebi nije ugnežden.

Ugneždena klasa može da

- 1) Bude izvedena iz proizvoljne klase
- 2) Implementira proizvoljan interfejs
- 3) Bude osnova za proširivanje
- 4) Se deklarise kao **final** ili **abstract**

Ime ugneždenog tipa je direktno dostupno u obuhvatajućem tipu, a izvan mu se pristupa kvalifikacijom **<ObuhvatajućiTip>.<UgneždeniTip>**

Kao član obuhvatajuće klase, ugneždena klasa ima pristup svim članovima obuhvatajuće klase, čak iako su deklarirani kao privatni (ovo važi i u obrnutom smeru, obuhvatajuća može da pristupi članovima unutrašnje). Obuhvatajuća klasa takođe ima potpuni pristup članovima ugneždene klase.

static ugneždeni tip	nestatički ugneždeni tip
<pre>class Spoljasnja { // detalji spolj. klase static class Ugneždjena { // detalji ugneždene klase } }</pre>	<pre>class Spoljasnja { // detalji spolj. klase class Unutrasnja { // detalji unutrasnje klase } }</pre>
ugneždjena klasa deklarirana kao static se naziva statičkom ugnežđenom klasom (static nested class)	Nestatička ugneždjena klasa se naziva unutrašnjom klasom (inner class)
Statički ugneždeni tipovi služe kao mehanizam struktuiranja tipova. Objekti ugnežđenih tipova su nezavisni od objekata obuhvatajuće klase.	Omogućavaju definisanje posebnog odnosa zavisnosti između njihovih objekata i objekata spoljašnje klase. Objekat unutrašnje klase je uvek u vezi sa jednim objektom spoljašnje klase. Objekat spoljašnje klase može da bude u vezi sa više objekata unutrašnje klase

static ugneždeni tip	nestatički ugneždeni tip
Statička ugneždjena klasa ne može direktno (imenovanjem bez kvalifikacije) da pristupa nestatičkim poljima ili metodima obuhvatajuće klase (može preko reference na objekat) Klasa ugneždjena u interfejsu je podrazumevano statička Ugneždeni interfejs je podrazumevano statički	Unutrašnja klasa može direktno da pristupa nestatičkim članovima spoljašnje Unutrašnja klasa ne može da sadrži statičke članove (izuzev finalnih statičkih polja inicijalizovanih konstantnim izrazom)
<pre>Spoljasnja.Unutrasnja su = Spoljasnja.new Unutrasnja();</pre>	<pre>Spoljasnja s = new Spoljasnja(); // kreiranje objekta spoljasnje Spoljasnja.Unutrasnja su = s.new Unutrasnja(); // kreiranje objekta unutrasnje spolja this.Unutrasnja su = this.new Unutrasnja(); // kreiranje objekta u spoljsnjoj klasi</pre>

PRIMER

```
public class RacunUbanci {
    private long broj; private long stanje; private Akcija poslednjaAkcija;
    public class Akcija {
        private String akcija; private long iznos;
        Akcija (String akcija, long iznos) {
            this.akcija=akcija; this.iznos=iznos;}
        public String toString() {
            return broj + ": " + akcija + " " + iznos;}
    }

    public void uplata (long iznos){
        stanje+=iznos;
        poslednjaAkcija=new Akcija("uplata", iznos);}

    public void isplata(long iznos){
        stanje-=iznos;
        poslednjaAkcija=new Akcija("isplata", iznos);}

    public void prenos(RacunUbanci drugi, long iznos){
        drugi.isplata(iznos);
        uplata(iznos);
        poslednjaAkcija=this.new Akcija("prenosna",iznos);
        drugi.poslednjaAkcija=drugi.new Akcija("prenosna", iznos);}
}
```

Unutrašnja klasa može da pristupi bez kvalifikovanja članu spoljašnje, dok spoljašnja klasa može da pristupi članu unutrašnje samo preko kvalifikacije (imena objekta)

34. Događaji. Objasniti ceo mehanizam. Listener-i i adapterske klase. Dati primer upotrebe

Događaj predstavlja signal koji se šalje programu kao informacija o tome da se nešto desilo. Događaj se generiše kao akcija od strane korisnika ili kao signal koji je poslat od strane OS-a.

Java događaje:

- 1) **Generišu** izvori (sources) događaja, a
- 2) **Obrađuju** objekti klasa osluškivača (listeners) događaja

Izvori događaja generišu objekat tipa događaj i šalju osluškivačima. Svaki izvor događaja ima sopstvenu listu osluškivača i posebnu listu za svaku vrstu događaja. Da bi se neki osluškivač našao u listi nekog izvora mora se **registrovati** kod izvora.

Definisanje obrade događaja:

- 1) **Definisanje osluškivača**
- 2) **Registracija osluškivača kod izvora**

Definisanje klase osluškivača:

- 1) **Klasa mora da implementira interfejs nekog osluškivača**
- 2) **Klasa mora da proširuje neku klasu koja implementira interfejs osluškivača**

```
public class Obrada implements ActionListener {...}
```

Mora se implementirati metod koji prepisuje interfejs osluškivača

```
public void actionPerformed(ActionEvent) { /*...*/ }
```

Registrowanje instance klase osluškivača događaja kod izvora

```
komponentaIzvor.addActionListener(obradjivac);
```

Primer:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class TestButton {
    public static void main(String[] args){
        ButtonFrame frame = new ButtonFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
class ButtonFrame extends JFrame {
    public ButtonFrame()
    {
        setTitle("Test s bojom pozadine");
        setSize(300,200);
        Container cp = getContentPane();
        ButtonPanel panel = new ButtonPanel();
        cp.add(panel);
    }
}
class ButtonPanel extends JPanel implements ActionListener
{
    JButton yellow;
    JButton blue;
    JButton red;
    public ButtonPanel()
    {
        yellow = new JButton("Yellow");
        blue = new JButton("Blue");
        red = new JButton("Red");
        add(yellow);
        add(blue);
        add(red);
        yellow.addActionListener(this);
        blue.addActionListener(this);
        red.addActionListener(this);
    }
    public void actionPerformed(ActionEvent event) {
        Object obj = event.getSource();
        if(obj.equals(yellow)) setBackground(Color.YELLOW);
        else if(obj.equals(blue)) setBackground(Color.BLUE);
        else if(obj.equals(red)) setBackground(Color.RED);
    }
}
```

V. Niti

35. Niti

Nit je skup koraka/ komandi koji se izvršavaju sekcencijalno/redom. Predstavlja jedno izvršenje nekog dela koda unutar adresnog prostora nekog procesa kreiranog unutar OS-a. Više niti može biti kreirano unutar istog procesa i obično se kreiraju nad pozivom jednog potprograma.

Niti u okviru istog procesa imaju zajednički **adresni prostor, programski kod i resurse**, a imaju sopstvenu **poziciju u izvršavanju i lokalne podatke za potprograme**

36. Višenitno programiranje

Višenitno programiranje je simultano izvršavanje više niti koje mogu da dele zajedničke resurse.

Višenitni program može da se izvršava **konkurentno** (ne pretpostavlja više procesora) i **paralelno** (pretpostavlja više procesora koji omogućavaju stvarni paralelizam).

Primeri upotrebe: tekst procesor, web browser, server

37. Java niti

Niti su definisane klasom **Thread**.

Aktivan objekat se kreira komandom **Thread nit = new Thread();**

Nakon kreiranja niti, ona se može **konfigurisati** (postavljanje prioriteta, imena,...) i **pokrenuti** (pozivom start metode). Metod start aktivira novu nit kontrole (nit postaje spremna), a zatim se metod završava i to tako što pokreće njen **run** metod

```

public class TestThread{
    public static void main (String[] args) {
        PingPong nit1 = new PingPong("ping", 33); // 1/30s
        PingPong nit2 = new PingPong("PONG", 100); // 1/10s
        nit1.start();
        nit2.start();
    }
}
class PingPong extends Thread {
    public String rec;
    int vreme; PingPong(String r, int v){
        rec=r; vreme=v;
    }
    public void run(){
        try{
            for(int i=0;i<30;i++){
                System.out.println(rec);
                sleep(vreme);
            }
        } catch (InterruptedException e){ return; }
    }
}

```

nit je kreirana – stanje new

nit je startovana, tj. pokrenut je njen metod run – stanje runnable/running
u zavisnosti od toga da li se trenutno izvršava na procesoru ili čeka na izvršavanje

nit je završila sa radom – stanje dead

38. Runnable objekti

Drugi način kreiranja niti je implementacijom **interfejsa Runnable**. Interfejs Runnable je apstrakcija koncepta aktivnih objekta – objekta koji izvršava neki kod konkurentno sa drugim takvim objektima. Klasa Thread implementira Runnable. Interfejs Runnable ima samo metod **run()**

```

public class TestThread{
    public static void main (String[] args) {
        new Thread(new PingPong("ping", 33)).start(); // 1/30s
        new Thread(new PingPong("PONG", 100)).start(); // 1/10s
    }
}
class PingPong implements Runnable {
    public String rec;
    int vreme; PingPong(String r, int v){
        rec=r; vreme=v;
    }
    public void run(){
        try{
            for(int i=0;i<30;i++){
                System.out.print(rec+"\n");
                Thread.sleep(vreme);
            } catch (InterruptedException e){
                return;
            }
        }
    }
}

```

definisanje

prvo kreirati objekat koji jeste Runnable, tj. koji ima ponašanje definisano u run metodu
a zatim kreirati nit koja će "preuzeti" njegovo ponašanje i osobine

životni ciklus

je isti kao i kod niti koje su definisane kao proširenja klase Thread

InterruptedException is thrown when a thread is waiting, sleeping, or otherwise paused for a long time and another thread interrupts it using the interrupt method in class Thread.

39. Sinhronizacija

Sinhronizacija niti obezbeđuje **ekskluzivan pristup podacima** samo jednoj niti kada se ona nalazi u svom kritičnom regionu.

40. Java katanaci

Java katanaci predstavljaju implementaciju koncepta monitora i svaki objekat poseduje jedan **implicitni katanac**. Kada nit dobije katanac od objekta, tada samo ta nit može da pristupa objektu što znači da ako za to vreme druga nit zatraži katanac istog objekta, blokiraće se i moći će da mastavi sa svojim radom tek kada prva nit oslobodi katanac.

Zahtevanje katanca se postiže navođenjem ključne reči **synchronized**.

```

public class Location {
    private double x,y;
    Location(double x, double y) {this.x=x; this.y=y;}

    synchronized double x() {return x;}
    double y() {
        synchronized(this){
            return y;
        }
    }
}

```

ono što se zaključava jeste objekat čiji je sinhronizovani metod pozvan

41. Sinhronizacija statičkih promenljivih

Zajedničke metode klase rade nad bravom klase, a ne nad bravom objekta. Dve niti ne mogu da izvršavaju **sinhronizovane statičke metode** nad istom klasom u isto vreme. Brava klase nema efekat nikakav efekat na objekte te klase, a to znači da jedna nit može da izvršava sinhroni statički, dok druga može da izvršava sinhronizovani nestatički metod.

42. Sinhronizacija i nasleđivanje

Kada se redefiniše metod u izvedenoj klasi, **osobina synchronized neće biti nasleđena**, a redefinisani metod može biti sinhronizovan ili ne bez obzira na odgovarajući metod roditeljske klase. Novi nesinhronizovani metod neće ukinuti sinhronizovano ponašanje metoda roditeljske klase.

VI. OOP paradigma i UML

43. Pojmovi: programerska paradigma, programerska tehnika, programerski stil, programerska kultura

Programerska paradigma – obrazac koji služi kao učenje koje se sledi u procesu programiranja

Programerska tehnika – strategija rešavanja problema koja se primenjuje u algoritmu

Programerska stil – stil kojim je program napisan

Programerska kultura – sveukupan izraz jednog programera; definišu je glavne paradigme, stilovi i programerske tehnike koje jedan programer koristi ili kojima vlada

44. Osnovni principi objektno-orijentisanog modela

Osnovni: APSTRAKCIJA, KAPSULACIJA, MODULARNOST, HIJERARHIJA

Dodatni: TIPIZACIJA, KONKURENTNOST, PERZISTENCIJA

45. Apstrakcija

Apstrakcija je uprošćeni opis sistema kojim se naglašavaju samo neke osobine ili detalji. To je postupak kojim se ističu bitne karakteristike objekta i time definišu konceptualne granice. Postoji apstrakcija entiteta i akcije.

46. Kapsulacija

Kapsulacija je proces skrivanja onih elemenata apstrakcije koji definišu strukturu i ponašanje. Služi da razdvoji konceptualni interfejs od implementacije apstrakcije. Interfejs apstrakcije čini ono što je od nje dostupno spolja.

47. Modularnost

Modularnost je sposobnost sistema da se razloži na skup kohezivnih i slabo spregnutih modula. Moduli su fizičke jedinice koje predstavljaju komponente sistema i mogu se održavati nezavisno.

48. Hijerarhija

Hijerarhija je uređivanje apstrakcija. Imamo:

- **Nasledivanje** – hijerarhija određena relacijom *is-a*
- **Sadržanje** – hijerarhija određena relacijom *part-of*
- **Klasifikacija, generalizacija, specijalizacija**

49. Tipizacija

Tipizacija se odnosi na nemogućnost ili ograničenu mogućnost da se objekti različitih klasa međusobno razmenjuju. Imamo statičku i dinamičku tipizaciju (vreme kada se ime promenljive vezuje za tip)

50. Konkurentnost i perzistencija

Konkurentnost se odnosi na mogućnost definisanja aktivnih i pasivnih objekata. Aktivan objekat ima svoju nit kontrole

Perzistencija se odnosi na postojanje objekta kroz:

- **Vreme** – nastavlja da postoji i nakon nestanka njegovog stvaraoca
- **Prostor** – moguće premeštanje iz adresnog prostora u koje je stvoren

51. Pojmovi objektno-orijentisane metodologije razvoja softvera

Objektno-orijentisana analiza (OOA) – kreiranje modela funkcionalnih zahteva sistema. Potrebno je:

- 1) Naći i organizovati objekte
- 2) Opisati kako među sobom komuniciraju
- 3) Definirati ponašanje objekata
- 4) Definirati karakteristike objekata

Objektno-orijentisano modelovanje (OOD) obuhvata

- 1) proces dekompozicije,
- 2) notaciju za predstavljanje logičkih i fizičkih, statičkih i dinamičkih aspekata modela sistema koji se projektuje

Objektno-orijentisano programiranje (OOP)

Objektno-orijentisani jezik (OOL)

52. Model

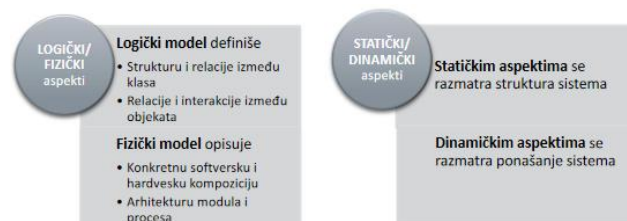
Model je pojednostavljena slika realnog sistema i predstavlja apstrakciju realnog sistema posmatranog iz određenog ugla. Njegova namena je da se sistem bolje razume

Ciljevi modeliranja:

- 1) Vizuelizacija
- 2) Specifikacija strukture sistema i ponašanja sistema
- 3) Dokumentovanje projektnih odluka
- 4) Smanjenje cene razvoja

53. OO Analiza i projektovanje

OO Analiza i projektovanje se obavljaju u terminima klasa, objekata, njihovih relacija i interakcija



54. UML

UML je standardni jezik za specifikaciju OO modela pomoću vizuelnih i grafičkih notacija. Koristi se za modelovanje i projektovanje softverskih sistema, naročito sistema pravljenih primenom objektno-orijentisanih tehnologija.

55. Elementi UML

- 1) Stvari – strukturne stvari, stvari ponašanja, stvari grupisanja, stvari označavanja
- 2) Relacije – zavisnost, asocijacija, generalizacija, realizacija
- 3) Dijagrami – klasa, objekata, slučajeva upotrebe, sekvence, kolaboracije, stanja, aktivnosti, komponenta i raspoređivanja

56. UML dijagram

Dijagram je grafička reprezentacija skupa povezanih elemenata. Predstavlja graf u kome su čvorovi stvari, a grane relacije.

Vrste dijagrama:

- 1) Dijagrami strukture – za prikaz statičkih/strukturnih aspekata sistema
- 2) Dijagrami ponašanja -za prikaz dinamičkih aspekata sistema

57. Dijagrami strukture

- 1) **Dijagram klasa** – najčešće korišćen u objektnom modeliranju. Prikazuje skup klasa interfejsa, saradnji povezanih relacijama. Specifira logičke i statičke aspekte modela. Većina alata podržava generisanje skeleta koda iz dijagrama klasa.
- 2) **Dijagram objekata** – skup objekata i njihovih veza
- 3) **Dijagram komponenta** – komponente, njihova struktura i zavisnost skupa komponenta
- 4) **Dijagram raspoređivanja** – konfiguracija čvorova obrade
- 5) **Dijagram paketa** – grupisanje elemenata u pakete

58. Dijagrami ponašanja

- 1) **Dijagrami slučajeva korišćenja** – skup slučajeva korišćenja, aktera i njihovih relacija
- 2) **Dijagrami interakcije** – skup uloga i njihovih veza sa porukama koje razmenjuju
 - **Dijagram sekvence** – vremenski redosled poruka
 - **Dijagram komunikacije** – strukturna organizacija povezanih uloga koje šalju i primaju poruke
- 3) **Dijagrami aktivnosti** – tok od jedne do druge aktivnosti u sistemu
- 4) **Dijagrami stanja** – konačni automat koji obuhvata stanja, tranzicije, događaje i aktivnosti

59. Stvari strukture

- 1) Klasa – osnovna jedinica strukturnog modela. Retko je izolovana jer uglavnom dobija smisao samo uz druge klase. Predstavlja opis skupa objekata koji dele iste *atribute, operacije, relacije i semantiku*.
- 2) Aktivna klasa – ima vlasitu nit kontrole
- 3) Interejs
- 4) Komponenta – modularni deo sistema koja ostvaruje realizaciju skupa interfejsa i sakriva implementaciju, realizuje određene interfejse i može se zameniti drugom koja realizuje iste
- 5) Artefakt – fizički i zamenljivi deo sistema, može biti fajl sa izvršnim izvornim kodom i sl.
- 6) Čvor – fizički element koji reprezentuje resurs obrade

60. Relacije

- 1) **Zavisnost** je semantička relacija između dve stvari u kojoj izmena jedne (nezavisne) stvari može uticati na semantiku druge (zavisne) stvari. Mogu postojati različite vrste zavisnosti kao što su: poziva, koristi, kreira.
- 2) **Generalizacija** je relacija specijalizacije/generalizacije u kojoj objekti specijalizovanog tipa (deca) dele strukturu i ponašanje generalizovanog elementa (roditelja)
- 3) **Asocijacija** je relacija između klasa čiji su objekti na neki način povezani. Ta veza između objekata klasa postoji određeno duže vreme, a ne samo tokom trajanja izvršavanja operacije jednog objekta koji poziva drugi objekat. Instanca asocijacije se naziva *vezom* i postoji između objekata datih klasa. Svaka strana u asocijaciji ima svoju ulogu koja se može naznačiti na strani same klase.
- 4) **Realizacija** je semantička relacija između klasifikatora u kojoj jedan klasifikator specifikira ugovor, a drugi garantuje njegovu implementaciju.

Primer: Relacija između interfejsa i klasa ili komponenata koje ga realizuju

61. Multiplikativnost

Multiplikativnost – na jednoj strani asocijacije označava se broj objekata sa te strane koji su u vezi sa tačno jednim objektom sa druge strane relacije. Može biti:

- 1) n – tačno n
- 2) $*$ - proizvoljno mnogo uključujući nijedan
- 3) $0..3$ – opseg

62. Agregacija

Agregacija je vrsta asocijacije kod koje jedna strana igra ulogu **celine**, a druga ulogu **dela**.

Celina sadrži delove - **has-a** relacija.

Agregacija ne govori ništa o uzajamnom odnosu životnih vekova celine i dela. Pod agregacijom se podrazumeva **deljena** ili **slaba forma agregacija** jer deo jedne agregacije može biti deo još neke agregacije. Dakle, deo je nezavisan, pa se uništenjem celine ne uništava deo.

63. Kompozicija

Kompozicija je asocijacija u kojoj postoji odnos celina/deo, ali je celina odgovorna za životni vek dela. Deo kompozicije je deo samo jedne celine, Objekat dela nastaje u toku života celine i može biti uništen pre kraja životnog veka celine. Ako se uništi celina, uništava se i deo. Kompozicija predstavlja **jaku formu agregacije**.

64. Navigabilnost

Navigabilnost predstavlja dostupnost objekta jedne klase iz objekta druge klase koja je sa prvom u nekoj relaciji. Za asocijaciju bez ukrasa navigabilnosti se smatra da je navigabilnost *neodeđena*. Krstić označava da nema navigabilnosti prema označenoj strani. Strelica označava navigabilnost u naznačenom pravcu.